

MATH 4740 Final Project

Kaine Griffin

University of Georgia, Athens, GA 30602, USA (e-mail:
kmg57191@uga.edu).

Abstract In this project we investigate simple control over a reparable system that describes the probability of a device in good or failure mode. Given the failure and repair rates, $\lambda_0 = 0.2$ and $\mu_1 = 0.8$ respectively, and desired probabilities of good mode, $p_0^*(t) = 0.9$, and failure mode, $p_1^*(t) = 0.1$, we optimize a control function, $u(t) \geq 0$ that penalizes deviations from the desired probabilities with parameters γ_0, γ_1 .

We discretize the system using backwards Euler's method. Then, a gradient descent algorithm is implemented and Lagrange multipliers are utilized to backwards propagate the gradient of our objective function, J with respect to u . We find the problem becomes a quadratic programming problem in the case of $\gamma_1 = 0$. When $\gamma_1 > 0$, we find $u(t)$ becomes intermittent.

Keywords: Optimal control, gradient descent, reparable systems, Lagrange multipliers

1. PROBLEM DESCRIPTION

We look at a simple reparable system, with given repair and failure rates, and determine the probability a device is in good or failure mode. We aim to find an optimal control function, $u(t) \geq 0$, such that we get as close to a desired probability in this system. The system is as follows:

$$\begin{cases} \frac{dp_0(t)}{dt} = -\lambda_0 p_0(t) + \mu_1 p_1(t) + u(t) \\ \frac{dp_1(t)}{dt} = -\mu_1 p_1(t) + \lambda_0 p_0(t) - u(t) \end{cases} \quad (1)$$

with $\lambda_0 = 0.2$, $\mu_1 = 0.8$ and initial conditions

$$p_0(0) = 1, \quad p_1(0) = 0. \quad (2)$$

Here,

- (1) $p_0(t)$: Probability that the device is in good mode at time t
- (2) $p_1(t)$: Probability that the device is in failure mode at time t
- (3) $u(t)$: nonnegative control input

We can write the system into matrix-vector form:

$$\begin{bmatrix} \frac{dp_0}{dt} \\ \frac{dp_1}{dt} \end{bmatrix} = \begin{bmatrix} -0.2 & 0.8 \\ 0.2 & -0.8 \end{bmatrix} \begin{bmatrix} p_0(t) \\ p_1(t) \end{bmatrix} + \begin{bmatrix} 1 \\ -1 \end{bmatrix} u(t)$$

$$A = \begin{bmatrix} -0.2 & 0.8 \\ 0.2 & -0.8 \end{bmatrix} \quad B = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

We define our objective function that we want to minimize

$$J(p_0(t); p_1(t); u(t)) = \frac{1}{2} \|p_0(t) - p_0^*\|_2^2 + \frac{1}{2} \|p_1(t) - p_1^*\|_2^2 + \frac{\gamma_0}{2} \|u(t)\|_2^2 + \gamma_1 \|u(t)\|, \quad (P)$$

which is subject to (1), has initial conditions (2), and where $\gamma_0, \gamma_1 \geq 0$ are control weight parameters.

We discretize the system of ordinary differential equations with backward Euler's method. We find that

$$\begin{aligned} \frac{\begin{bmatrix} p_{0,i+1} - p_{0,i} \\ p_{1,i+1} - p_{1,i} \end{bmatrix}}{\Delta t} &= A \begin{bmatrix} p_{0,i+1} \\ p_{1,i+1} \end{bmatrix} + B u_{i+1} \\ \begin{bmatrix} p_{0,i+1} \\ p_{1,i+1} \end{bmatrix} &= \Delta t A \begin{bmatrix} p_{0,i+1} \\ p_{1,i+1} \end{bmatrix} + \Delta t B u_{i+1} + \begin{bmatrix} p_{0,i} \\ p_{1,i} \end{bmatrix} \\ (I - \Delta t A) \begin{bmatrix} p_{0,i+1} \\ p_{1,i+1} \end{bmatrix} &= \Delta t B u_{i+1} + \begin{bmatrix} p_{0,i} \\ p_{1,i} \end{bmatrix} \\ \begin{bmatrix} p_{0,i+1} \\ p_{1,i+1} \end{bmatrix} &= (I - \Delta t A)^{-1} \left(\Delta t B u_{i+1} + \begin{bmatrix} p_{0,i} \\ p_{1,i} \end{bmatrix} \right) \end{aligned} \quad (3)$$

for $i = 0, \dots, 99$, $\Delta t = \frac{1}{20}$ and with $p_{0,0} = 1$ and $p_{1,0} = 0$. We define

$$C = (I - \Delta t A)^{-1} = \begin{bmatrix} 104 & 4 \\ 105 & 105 \\ 1 & 101 \\ 105 & 105 \end{bmatrix}$$

which is the Jacobian, $\frac{\partial p_{i+1}}{\partial p_i}$

$$P_0 = [p_{0,1}, p_{0,2}, \dots, p_{0,100}]^T$$

$$P_1 = [p_{1,1}, p_{1,2}, \dots, p_{1,100}]^T$$

$$\mathbf{u} = [u_1, u_2, \dots, u_{100}]^T$$

which allows us to rewrite (P) as

$$\min_{\mathbf{u}} J(P_0; P_1, \mathbf{u}) = \left\| \begin{bmatrix} P_0 \\ P_1 \end{bmatrix} - \begin{bmatrix} P_0^* \\ P_1^* \end{bmatrix} \right\|_2^2 + \frac{\gamma_0}{2} \|\mathbf{u}\|_2^2 + \gamma_1 \|\mathbf{u}\|_1. \quad (P^*)$$

subject to (3). We will use LaGrange multipliers and a gradient descent step to minimize $J(P_0; P_1; \mathbf{u})$ with respect to $\mathbf{u}$.

2. METHODOLOGY

We construct a gradient descent algorithm to optimize our control vector $\mathbf{u}$. First, we find $\nabla_{\mathbf{u}} J(P_0; P_1; \mathbf{u})$

$$\begin{aligned}\nabla_{\mathbf{u}} J(P_0; P_1; \mathbf{u}) &= \frac{d}{d\mathbf{u}} \left[\frac{\gamma_0}{2} \left(\sum_{i=1}^{100} |u_i|^2 \right)^2 + \gamma_1 \sum_{i=1}^{100} |u_i| \right] \\ &= \gamma_0 \mathbf{u} + \gamma_1 \text{sgn}(\mathbf{u}).\end{aligned}$$

A sign function is used here since $\mathbf{u} \geq 0$, $\forall u \in \mathbf{u}$. We define the following algorithm:

Algorithm 1 Gradient Computation

Initialize $\lambda_{100} = [0, 0]^T \triangleright$ Final condition for Lagrange multipliers
for $i = 99, \dots, 0$ **do**
 Compute $\nabla_{P_0} J^{(i+1)} = P_0^{(i+1)} - P_0^*$
 Compute $\nabla_{P_1} J^{(i+1)} = P_1^{(i+1)} - P_1^*$
 $\lambda_i = C^T \lambda_{i+1} + \begin{bmatrix} \nabla_{P_0} J^{(i+1)} \\ \nabla_{P_1} J^{(i+1)} \end{bmatrix} \triangleright$ update λ_i
 $\nabla_{\mathbf{u}} J^{(i)} = \lambda_{i+1}^T (C \Delta t B)$
 $\nabla_{\mathbf{u}} J^{(i)} += \gamma_0 u_i + \gamma_1 \text{sign}(u_i) \triangleright$ Add control terms

We can now implement a gradient descent algorithm to minimize J with respect to $\mathbf{u}$.

Algorithm 2 Gradient Descent Algorithm

Initialize $\mathbf{u} = \mathbf{0}$, $\mathbf{v} = \mathbf{u}$
Parameters: $\alpha = 0.01$, $TOL = 10^{-6}$, $N = 100$
for $i = 0, \dots, N-1$ **do**
 Find P_0, P_1 using backwards Euler's method (3)
 Compute $\nabla_{\mathbf{u}} J^{(i)}$ with Algorithm 1
 Set $\mathbf{u} = \max\{\mathbf{u} - \alpha \nabla_{\mathbf{u}} J^{(i)}, \mathbf{0}\}$
 if $\|\mathbf{u} - \mathbf{v}\| \leq TOL$ **then**
 break
 $\mathbf{v} = \mathbf{u}$
return $\mathbf{u}$

We set $\mathbf{u}$ to $\max\{\mathbf{u} - \alpha \nabla_{\mathbf{u}} J, \mathbf{0}\}$ to enforce non-negativity. This is better than enforcing an absolute value since the shape of the data is preserved. We also see that Algorithm 2 has a time complexity of $O(n)$.

These algorithms were implemented using Numpy (Harris *et al.*, 2020) and results were plotted with Matplotlib (Hunter, 2007). Calculations were timed using the builtin `%%timeit` command in Jupyter notebooks. It runs a given cell of the Jupyter notebook seven times and reports the average CPU computation time and the standard deviation. All time comparisons were performed on a Intel i5-9600KF, 3701 Mhz with 32 GB of RAM on Windows 10.

3. RESULTS

3.1 Analytical Solution for $\gamma_1 = 0$

Let $\gamma_1 = 0$. We find that $(\mathbf{P}^*)$ becomes

$$\begin{aligned}\min_{\mathbf{u}} \quad & J(P_0; P_1; \mathbf{u}) = \left\| \begin{bmatrix} P_0 \\ P_1 \end{bmatrix} - \begin{bmatrix} P_0^* \\ P_1^* \end{bmatrix} \right\|_2^2 + \frac{\gamma_0}{2} \|\mathbf{u}\|_2^2 \\ \text{subject to} \quad & \begin{bmatrix} p_{0,i+1} \\ p_{1,i+1} \end{bmatrix} = C \left(\Delta t B u_{i+1} + \begin{bmatrix} p_{0,i} \\ p_{1,i} \end{bmatrix} \right)\end{aligned}$$

Let $\mathbf{P} = [P_0; P_1]$ and $\mathbf{P}^* = [P_0^*; P_1^*]$. We can rewrite our problem as

$$\begin{aligned}\min_{\mathbf{u}} \quad & J(\mathbf{P}, \mathbf{u}) = \|\mathbf{P} - \mathbf{P}^*\|_2^2 + \frac{\gamma_0}{2} \|\mathbf{u}\|_2^2 \\ \text{subject to} \quad & \mathbf{P} = D\mathbf{u} + \mathbf{h}\end{aligned}$$

where

$$\begin{aligned}(1) \quad D &= \begin{bmatrix} C \Delta t B & 0 & \dots & 0 \\ C^2 \Delta t B & C \Delta t B & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ C^{100} \Delta t B & C^{99} \Delta t B & \dots & C \Delta t B \end{bmatrix} \\ (2) \quad h &= \begin{bmatrix} C \\ 0 \\ C^2 \\ 0 \\ \vdots \\ C^{100} \\ 0 \end{bmatrix}\end{aligned}$$

these contain the recurrence relation found through Euler's method.

Now, we substitute our new expression for $\mathbf{P}$ get turn J into a function of $\mathbf{u}$.

$$\begin{aligned}J(\mathbf{u}) &= \|D\mathbf{u} + \mathbf{h} - \mathbf{P}^*\|_2^2 + \frac{\gamma_0}{2} \mathbf{u}^T \mathbf{u} \\ &= (D\mathbf{u} + \mathbf{h} - \mathbf{P}^*)^T (D\mathbf{u} + \mathbf{h} - \mathbf{P}^*) + \frac{\gamma_0}{2} \mathbf{u}^T \mathbf{u} \\ &= \mathbf{u}^T D^T D \mathbf{u} + 2\mathbf{u}^T D^T (\mathbf{h} - \mathbf{P}^*) + \|\mathbf{h} - \mathbf{P}^*\|_2^2 + \frac{\gamma_0}{2} \mathbf{u}^T \mathbf{u} \\ &= \mathbf{u}^T (D^T D + \frac{\gamma_0}{2} I) \mathbf{u} + 2\mathbf{u}^T D^T (\mathbf{h} - \mathbf{P}^*) + \|\mathbf{h} - \mathbf{P}^*\|_2^2 \\ &= \mathbf{u}^T (D^T D + \frac{\gamma_0}{2} I) \mathbf{u} + 2(\mathbf{h} - \mathbf{P}^*)^T D \mathbf{u}\end{aligned}$$

We drop the constant term since it is just a vertical shift and we see that the problem is now in a general quadratic programming problem form.

$$\begin{aligned}\min_{\mathbf{u}} \quad & J(\mathbf{u}) = \frac{1}{2} \mathbf{u}^T Q \mathbf{u} + \mathbf{c}^T \mathbf{u} \\ \text{subject to} \quad & D\mathbf{u} = (\mathbf{h} - \mathbf{P}^*) \\ \text{where} \quad & Q = D^T D + \frac{\gamma_0}{2} I \quad \mathbf{c}^T = 2D^T (\mathbf{h} - \mathbf{P}^*)\end{aligned}$$

Thus our optimal solution is in the form:

$$\mathbf{u}^* = Q^{-1} D^T (DQ^{-1} D^T)^{-1} \mathbf{c}^T$$

From this, we can infer that our discretized $\mathbf{u}$ vectors will be parabolic.

3.2 Numerical Results

In each of the figures shown, the dotted lines are the desired good/failure probabilities, 0.9 and 0.1.

In Figure 1, we find that for $N = 100$, when $\gamma_1 = 0$, the graphs are parabolic in shape as predicted; for $\gamma_1 = 1$, there are intervals where $u = 0$ which is a result of the way non-negativity was enforced in Algorithm 2. We can see that the remnants of a parabolic shape are still present in each of the γ_1 cases across all N .

4. OBSERVATION AND CONCLUSIONS

We look at the case of $\gamma_0 = \gamma_1 = 1$ in Figures (2-4) since it seems unchanged with even its intermittent drops

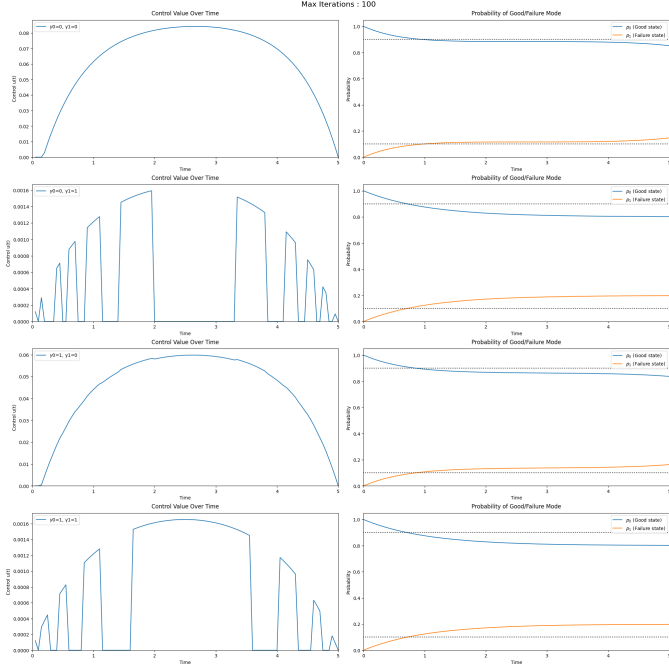


Figure 1. Control $u(t)$ and its associated P_0 and P_1 for $N = 100$

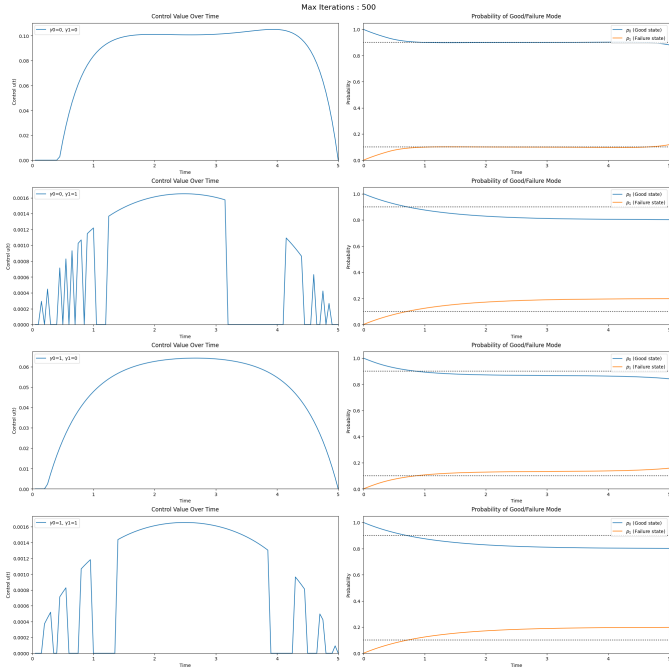


Figure 2. Control $u(t)$ and its associated P_0 and P_1 for $N = 500$

preserved. This is in contrast to the $\gamma_0 = 0, \gamma_1 = 1$ case which also has intermittent drops but it smooths out as N increases. From this, we conclude that the gradient descent algorithm does not converge when $\gamma_0 = \gamma_1 = 1$ and it converges slowly when $\gamma_0 = 0, \gamma_1 = 1$.

In the case of no control input, $\gamma_0 = \gamma_1 = 0$ changes as N increases. In 1, it is parabolic, but when N increases in 2, the initial increase is shifted to the right and there appears to be a local minima in the middle and only gets more distorted as N increases. Additionally, the probability vectors increasingly overfit to the desired probabilities

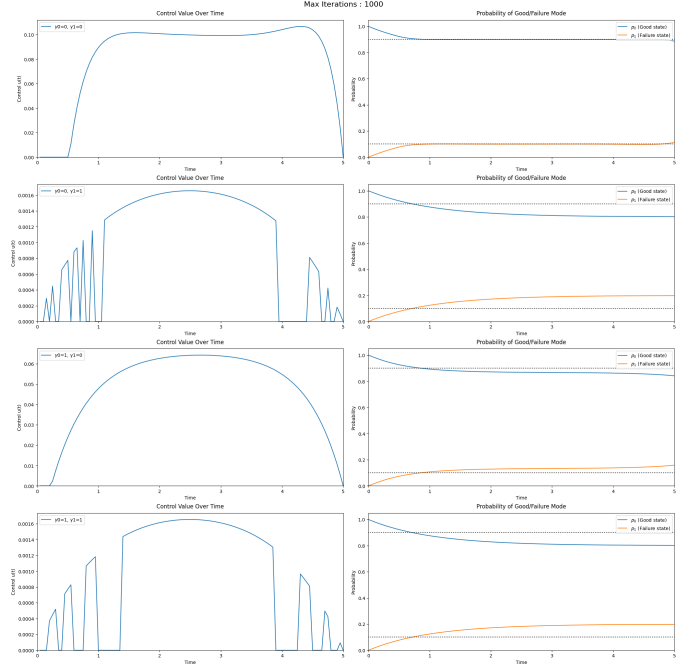


Figure 3. Control $u(t)$ and its associated P_0 and P_1 for $N = 1000$

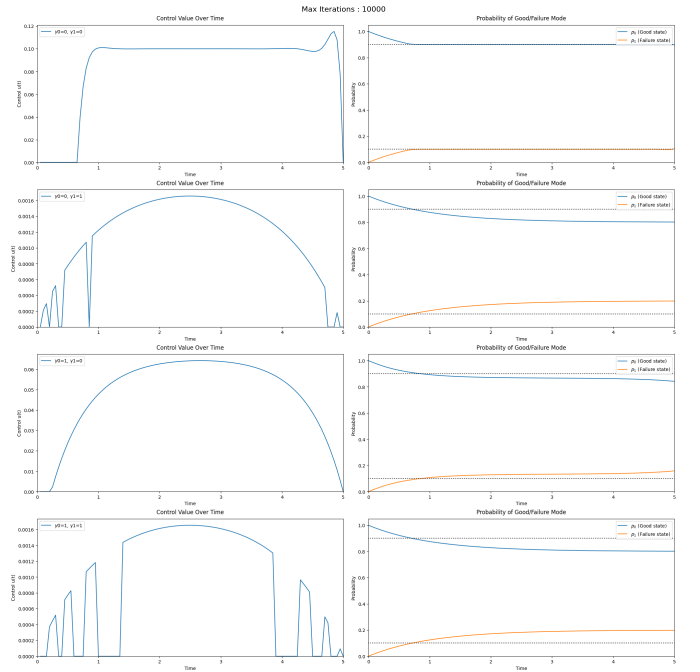


Figure 4. Control $u(t)$ and its associated P_0 and P_1 for $N = 10000$

with each N . The most stable solution occurs when $\gamma_0 = 1, \gamma_1 = 0$. We notice that at $N = 100$, it looks the same as when $\gamma_0 = \gamma_1 = 0$ but as N increases, the shape is not distorted - this is a result of the control input. We also see that the P_0 and P_1 found are close to the desired probabilities but they seem to diverge from the desired probability close to $t = 5$.

In order to easily go through each case of γ_0 and γ_1 , we implemented two for loops. Below is a table of computation times for various N s. We confirm that Algorithm 2 is $O(n)$.

N	$\bar{x}$	σ
100	604 ms	16.8 ms
500	3.03 s	76.9 ms
1000	5.33 s	108 ms
10000	45.1 s	1.03 s

Table 1. Computation Time Comparison

REFERENCES

- Harris, Charles R., K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke and Travis E. Oliphant (2020). Array programming with NumPy. *Nature* **585**(7825), 357–362.
- Hunter, J. D. (2007). Matplotlib: A 2d graphics environment. *Computing in Science & Engineering* **9**(3), 90–95.

APPENDIX

```

1 import numpy as np;
2 from matplotlib import pyplot as plt
3 #--- Constants and Givens ---#
4 del_t = 0.05
5 T = np.linspace(0,5,101)
6 A = np.array([[ -0.2, 0.8],
7               [ 0.2, -0.8]])
8 B = np.array([[1.0], [-1.0]])
9 I = np.eye(2)
10 C = np.linalg.inv(I-del_t*A)
11
12
13 # --- Backwards Euler's Method --- #
14
15 # P(i+1) = inv(I-del_t*A)(del_tBu(i+1)+P(i))
16 def backwardsEuler(u):
17     P = np.zeros(shape=[2, 101])
18     P[:, 0] = np.array([1,0]).T # initial
19     # conditions
20     for i in range(100):
21         tBu = del_t*B*u[i]
22         p = P[:,i].reshape(2,1)
23         P[:, i+1] = (C @ (tBu + p)).ravel()
24     p0 = P[0, :]
25     p1 = P[1, :]
26     return [p0, p1]
27
28 # --- Cost Function --- #
29 def J(p0, p1, u, c0, c1):
30     l2u = 0.5*c0*np.sqrt(sum((u**2)))
31     l1u = c1*sum(np.abs(u))
32     p0Error = np.sqrt(sum((p0-0.9)**2))
33     p1Error = np.sqrt(sum((p1-0.1)**2))
34     return 0.5*(p0Error+p1Error)+l2u+l1u
35
36 # --- Gradient of J(p0, p1, u) --- #
37 def dJ(p0, p1, u, c0, c1):
38     dJ_du = np.zeros(100)
39
40     lam = np.zeros(shape=[2, 101])
41     lam[:, -1] = 0
42
43     for i in reversed(range(100)):
44         dJ_dp0 = (p0[i+1] - 0.9)

```

```

45         dJ_dp1 = (p1[i+1] - 0.1)
46
47         dp_next_dp_prev = C.T
48         lam[:, i] = dp_next_dp_prev @ lam[:, i+1]
49         + np.array([dJ_dp0, dJ_dp1])
50
51         dp_next_du = C @ (del_t*B)
52         dJ_du[i] = lam[:, i+1].T @ dp_next_du.
53         flatten()
54
55         dJ_du += c0*u + c1*np.sign(u)
56
57         return dJ_du
58
59 N = 200
60 TOL = 1e-6
61 alpha = 0.01
62 costs = []
63 u = np.zeros(100)
64 prev_u = np.copy(u)
65
66 for c0 in range(2):
67     # print(c0)
68     for c1 in range(2):
69         # print(c1)
70         cost_history = []
71         for n in range(N):
72             p0, p1 = backwardsEuler(u)
73             cost_history.append(J(p0, p1, u, c0,
74                                 c1))
75
76             dJ_du = dJ(p0, p1, u, c0, c1)
77             u = np.maximum(u - alpha*dJ_du, 0)
78             # print(u<0)
79
80             if np.linalg.norm(u-prev_u) <= TOL:
81                 break
82             prev_u = np.copy(u)
83
84             k = [c0, c1, p0, p1, u]
85             costs.append(k)
86
87 # --- Plotting --- #
88 fig, axs = plt.subplots(len(costs), 2, figsize
89                         =(20,20), constrained_layout=True)
90
91 for i in range(axs.size//2):
92     axs[i][0].plot(T[1:], costs[i][4], label=rf"$
93 \gamma$0={costs[i][0]}, $\gamma$1={costs[i]
94 [1]}")
95     axs[i][0].set_xlabel("Time")
96     axs[i][0].set_ylabel("Control u(t)")
97     axs[i][0].set_xlim(0,5)
98     axs[i][0].set_ylim(0)
99     axs[i][0].set_title("Control Value Over Time"
100 , pad=10)
101     axs[i][0].legend(loc='upper left')
102
103     axs[i][1].plot(T[0:], costs[i][2], label=r"$
104 $p_0$ (Good state)")
105     axs[i][1].plot(T[0:], costs[i][3], label=r"$
106 $p_1$ (Failure state)")
107     axs[i][1].axhline(y=0.9, color = 'black',
108                       linestyle='dotted')
109     axs[i][1].axhline(y=0.1, color = 'black',
110                       linestyle='dotted')
111     axs[i][1].set_xlabel("Time")
112     axs[i][1].set_ylabel("Probability")
113     axs[i][1].set_xlim(0,5)
114     axs[i][1].set_ylim(0)

```

```
104     axs[i][1].set_title("Probability of Good/  
Failure Mode", pad=10)  
105     axs[i][1].legend(loc='center right')  
106  
107 plt.suptitle("Optimization Results With Different  
Parameters", fontsize=16)  
108 plt.show()
```